A Machine Learning Framework for Content Analytics and Engagement Prediction in Streaming Platforms

Rohan Kaushik[1], Prateek Nayak[1]*, Himanshu Pandey[1], Krishna Pandey[1], Kartik Kumar[1], Sakshi Rajwar[2]
[1]Department of Computer Science and Engineering
HMR Institute of Technology and Management, Delhi, India
rohankaushik3399@gmail.com, prateeknayak@hmritm.ac.in, himanshupandey1810@gmail.com,
krishnavidhi3@gmail.com, kartik.kumar8251@gmail.com
[2]Department of Applied Science
HMR Institute of Technology and Management, Delhi, India
sakshirajwar24@gmail.com
*Corresponding author: prateeknayak@hmritm.ac.in

Abstract

This research project involves applying a data-driven method for content analysis and prediction of content performance using machine learning algorithms to a set of metadata publicly accessible by Netflix viewers. Before starting training machine learning models, it was necessary to perform preliminary data transformation and feature engineering to identify possible associations between various attributes of the content such as its type, genre, release patterns, and geographic distribution.

Given the lack of user engagement data, an engagement score proxy was created based on domain knowledge and exploratory data analysis with the consideration of several factors like recency, genre popularity, content length, and maturity rating.

Four supervised machine learning models were designed and analyzed for performance: Linear Regression, Polynomial Regression, Decision Tree, and Random Forest. The experiments have shown that more complex algorithms exhibit higher efficiency; specifically, the Random Forest model achieved the highest performance ($R^2 = 0.89$).

Additionally, the model trained during the project has been included into a web application where a user can input values for the content attributes and receive predicted engagement scores as the output.

The above-described approach creates a reproducible framework for further implementation with real data.

Keywords: Content Analytics, Machine Learning, Engagement Prediction

1. Introduction

The rise of streaming platforms has significantly transformed the way audiences access audio visual content. Platforms such as Netflix, Amazon Prime Video, and Disney+ have redefined content delivery through advanced recommendation systems and global distribution strategies. Among these, Netflix stands out due to its extensive content library and rich metadata, including genres, release years, ratings, and geographic origins [1]. Although detailed viewing statistics remain proprietary, publicly available metadata provides a valuable foundation for data-driven analysis.

Machine learning techniques have been widely applied in streaming platforms for tasks such as recommendation systems, customer segmentation, and churn prediction [2], [3], [4], [5]. However, limited research has focused on integrating predictive machine learning models with optimization techniques to support content strategy decisions using only publicly available data. The solution to fill this research gap can be achieved through this paper where a data-driven approach combining data pre-processing, EDA, and supervised machine learning with Netflix public data on Kaggle [6] is proposed. A proxy for the engagement score is developed to conduct predictive analysis when real user data is not available. Various machine learning models will be considered for comparison to achieve high accuracy in engagement prediction. The entire system will be integrated as a web application.

Some of the most significant contributions made by this research include the creation of a full pipeline for pre-processing and feature extraction on the metadata from Netflix, the formulation of a proxy model to compute the engagement score in the absence of viewer data, the comparison between different machine learning models for engagement predictions, and the development of a web application.

2. Background and Related Work

The application of machine learning in streaming platform analytics has been widely explored in areas such as recommendation systems, content classification, and predictive modeling. This section highlights the most relevant contributions.

2.1 Content Recommendation and Filtering

Collaborative filtering and matrix factorization techniques form the foundation of modern recommendation systems [7]. The Netflix Prize further advanced this field by demonstrating the effectiveness of ensemble approaches [8]. Additionally, content-based filtering methods, which rely on item attributes rather than user history, are particularly useful in cold-start scenarios [9], aligning with the proxy-based setting of this work.

2.2 Machine Learning for Media Analytics

Supervised learning models have been widely applied to predict content performance. Prior studies have used decision trees and ensemble methods for rating prediction and churn analysis, showing improved accuracy over traditional approaches [4], [10]. Nonlinear models such as polynomial regression and tree-based methods have proven effective in capturing complex relationships in media datasets [11].

2.3 Exploratory Analysis of Netflix Data

Existing studies on Netflix datasets primarily focus on exploratory analysis and sentiment modeling [14], [15]. However, these works do not integrate predictive modeling with optimization or provide deployment-based solutions.

Overall, while prior research has addressed individual aspects of streaming analytics, limited work combines machine learning with optimization in a unified framework, which this paper aims to address.

3. Data Description and Preprocessing

3.1 Dataset Source and Format

The dataset used in this study is the Netflix Movies and TV Shows dataset available freely on Kaggle [6]. The dataset is a collection of content data from the Netflix site and is formatted in comma-separated value (CSV) form, thus readily available for data processing libraries.

The original dataset consists of roughly 8,807 entries, each of which corresponds to a unique title within the Netflix collection. Each entry corresponds to twelve attribute variables, namely, *show_id*, *type*, *title*, *director*, *cast*, *country*, *date_added*, *release_year*, *rating*, *duration*, *listed_in*, and *description*. Collectively, these attributes describe the individual title in terms of its identity, origin, temporal placement, target audience, and genre. It should be noted that the data set includes only content-related information, and excludes any information on viewership, income, subscriber numbers, etc. In particular, this property complies precisely with the restriction used in this study, whereby no financial information can be used.

3.2 CSV Format and Data Structure

There are many benefits associated with using CSV form of dataset for data science processes. The tabular format is easily handled by data manipulation software like pandas in Python. In the table formed from the dataset, each row represents a particular title in Netflix while columns represent various attributes of Netflix products. There is a first row that contains the names of all the columns.

There are some structural aspects of the CSV dataset that deserve consideration. First, the *type* variable holds categorical values with a binary classification of a “Movie” and a “TV Show,” representing the initial classification of the Netflix catalog. Next, the duration variable holds different data in a mixed format: the movie’s duration is measured in minutes (“90 min”), whereas the TV show’s duration is measured in seasons (“3 Seasons”). In order to process such mixed variables, there must be individual parsing of each variable’s data type during the preprocessing step.

3.3 Data Preprocessing

There are numerous instances of raw data quality problems in publicly compiled data sets, including the Netflix Kaggle dataset. The following data quality problems have been identified and are solved through an elaborate data pre-processing pipeline.

1) Missing Value Treatment

After loading of the raw CSV file, a check for null values was performed on all columns. The columns that had the highest percentage of missing values were the *director* and *cast* columns, with around 30% and 10% missing, respectively. Since both of these variables could be valuable, even though they have high cardinality, and filling in missing values would just add noise to the data, it was decided to keep those rows with missing director values but fill in the director variable with the value “unknown”.

The “country” column had approximately 10% missing values. As “country of origin” is one of the predictors employed in the prediction algorithm, the missing values were filled with the most common value for “country of origin” (United States). An indicator variable was then generated to identify rows where the country was imputed. The columns “date added” and “rating” had a few missing values; the rows with missing “date added” and “rating” values were deleted from the dataset, as these were only 0.5% of the whole dataset.

2) Type Conversion and Parsing

The *date_added* variable, which was stored in string type in the month-day-year format, was converted to Python date-time variables and later broken down into numerical attributes for year added, month added, and day added.

The *duration* field necessitated type-based parsing. Conditional parsing logic was implemented to parse numbers from durations for movies (as integers representing minutes) and shows (as integers representing seasons), thereby generating two new numeric fields: *movie_duration_min* and *tv_show_seasons*.

3) Categorical Encoding

A number of columns needed to be converted into numeric data types in order to be used as input for machine learning algorithms. In the *type* column, Movie is coded as 0 while TV Show is coded as 1. In the *rating* column, which includes ratings assigned by audiences such as TV-MA, TV-14, TV-PG, PG-13, R, PG, G, and NR, ordinal encoding was done using a maturity scale.

The *listed_in* attribute, having multiple genres listed as a string, was one-hot encoded. All unique genre tags were collected, and a corresponding binary attribute was generated for each genre tag. The value was set to 1 if the genre tag occurred in the *listed_in* attribute of a particular observation and 0 otherwise. Thus, a sparse binary vector of genre tags was concatenated with the main features vector. Finally, the *country* attribute was frequency encoded by replacing the country name with the frequency of its appearance within the dataset. It was assumed that the countries which occurred more often corresponded to big markets with special features.

4) Feature Normalization

Numerical features with different units of measurement such as *release_year* (with a range of years between 1925 and 2021), *movie_duration_min* (with a range of time durations between less than 10 minutes and more than 300 minutes), and country names represented by their frequencies were standardized through min-max scaling such that all numerical values lie within the [0, 1] range.

3.4 Exploratory Data Analysis (EDA)

As part of this study, exploratory data analysis (EDA) was done as an extra step after data pre-processing and before building models. The rationale for performing EDA involved analyzing the structure and statistical properties of the dataset and using this information to create new features for modeling purposes.

1) Distribution of Content Type

The distribution of the "type" variable indicates that Netflix's movie database is mainly made up of films and not TV shows. This aligns with Netflix's history because the company was founded on films alone before including TV shows.

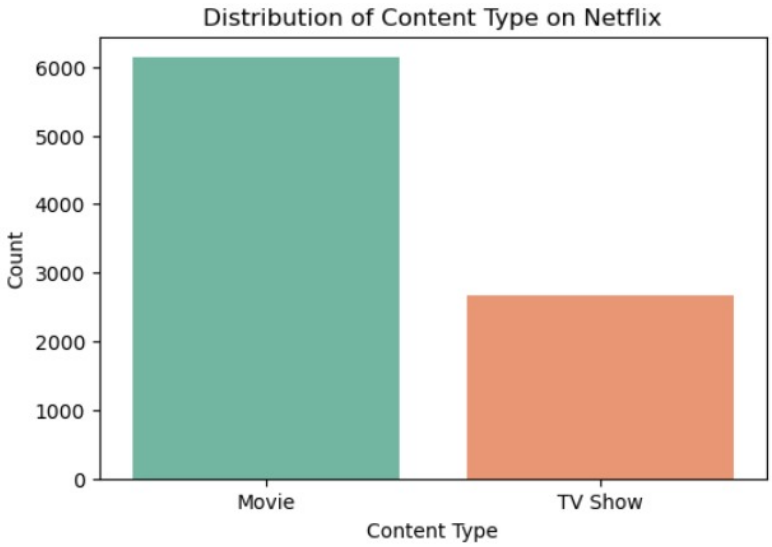


Fig. 1. Distribution of Content Type

2) Trends in Content Additions and Release Year

It is observed that there is a significant rise in content additions at Netflix following 2015, especially towards the end of 2019. The information provided is in reflection of the rapid increase of content streaming sites.

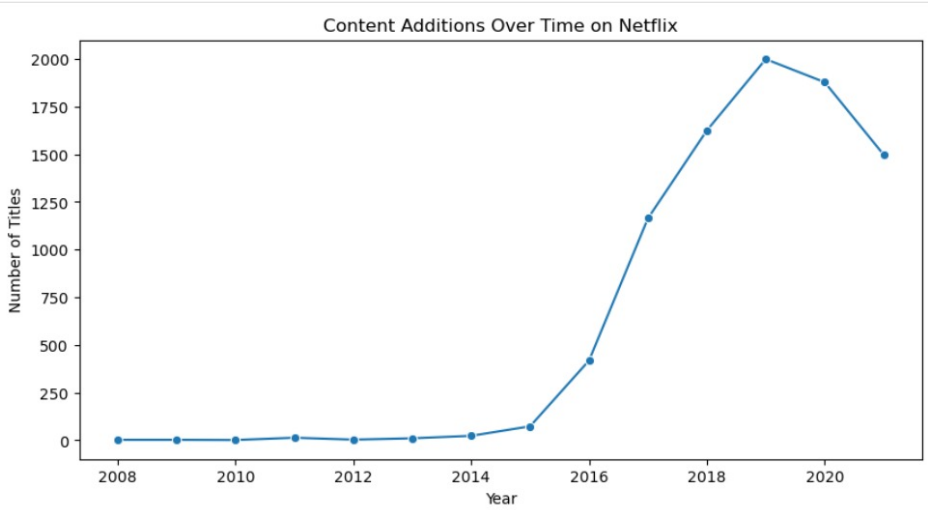


Fig. 2. Content Addition Over Time

3) Content Addition Geography

This specific dataset indicates that only a few countries engage in the process of adding content. They include, but not limited to the United States, India, and the United Kingdom.

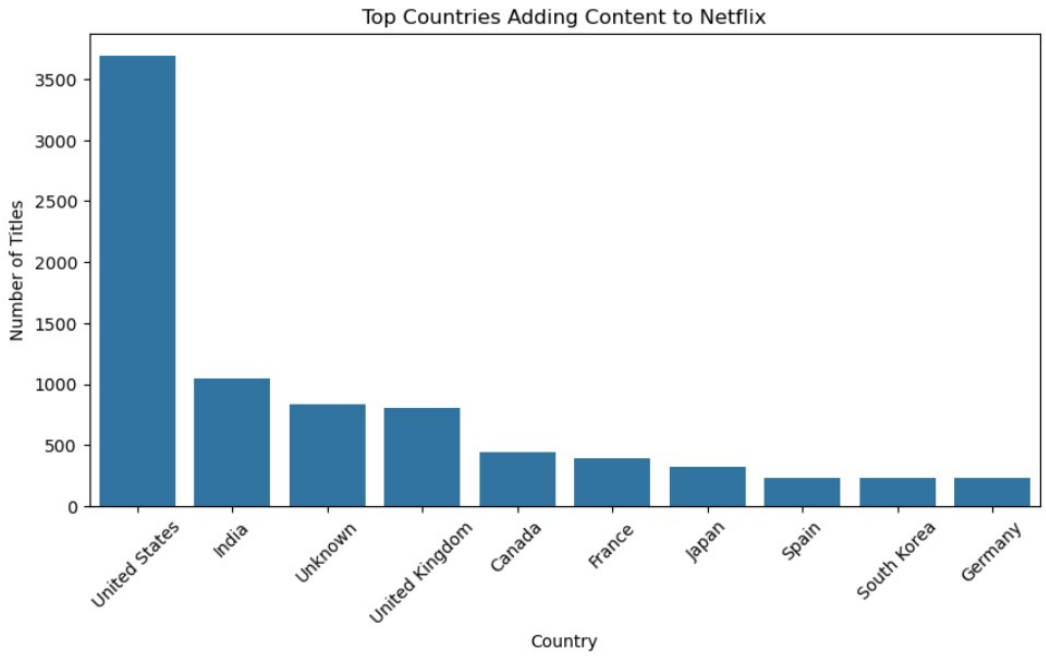


Fig. 3. Countries Adding Content to Netflix

4) Genre Count Analysis

Content genres analysis shows that Dramas, International Movies, and Comedies have the highest frequency rates compared to other content genres such as Documentaries and Stand-Up Comedy.

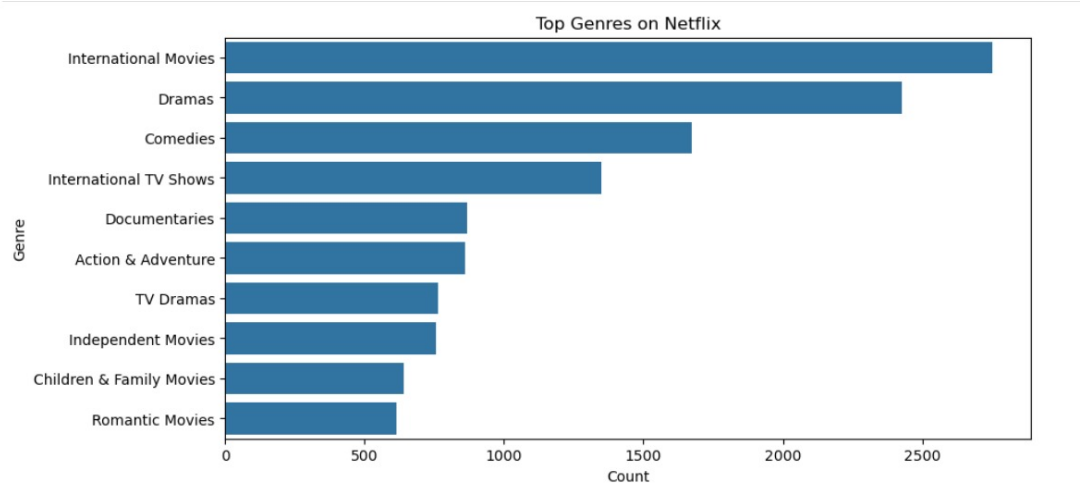


Fig. 4. Genre Count Analysis

5) Distribution of Rating Classifications

The rating categories show that there are more adult and teenager ratings compared to children's ratings; TV-MA and TV-14 ratings dominate in the dataset.

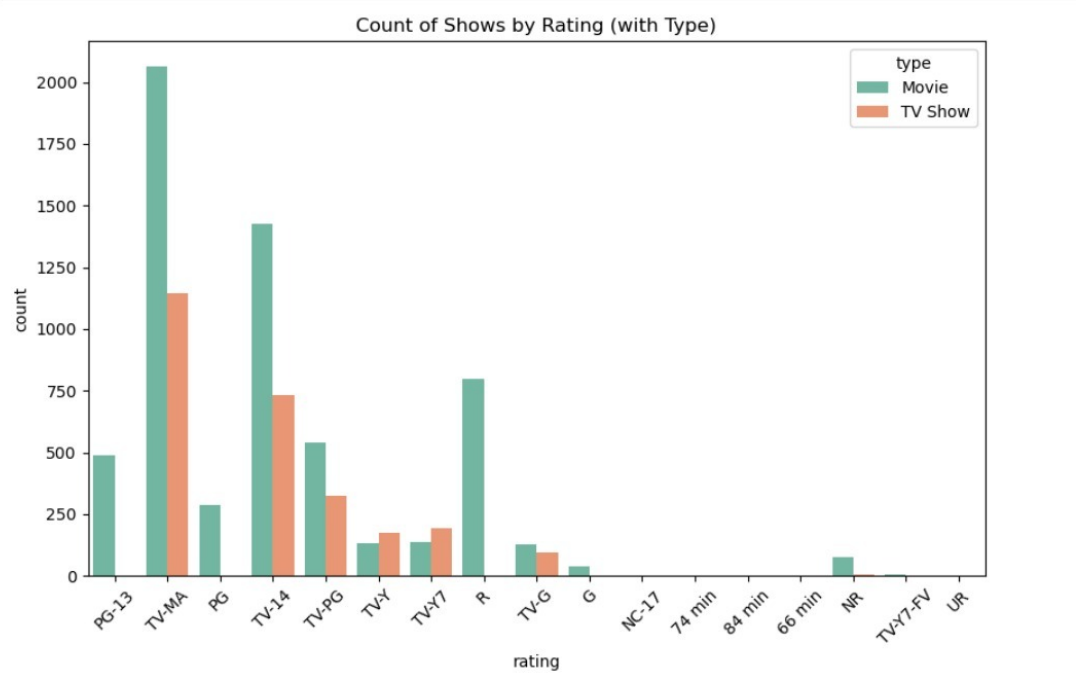


Fig. 5. Distribution of Rating Classifications

6) Duration Analysis

As seen in the analysis of duration, it is evident that the movies available on Netflix are mostly within the range of 75 to 125 minutes with a maximum of 100 minutes, consistent with the average duration of films. Conversely, most television series have just one season with the frequency rapidly decreasing with an increase in seasons.

7) Limitations of the Dataset

The dataset lacks any user engagement metrics like view count, watch count, and user ratings. This implies that the audience's behavior cannot be analyzed using this dataset.

8) Insights Obtained from EDA

EDA provides several important insights into the dataset, including:

(a) There were many additions of content after 2015.
(b) The USA and India are the top producers of content.

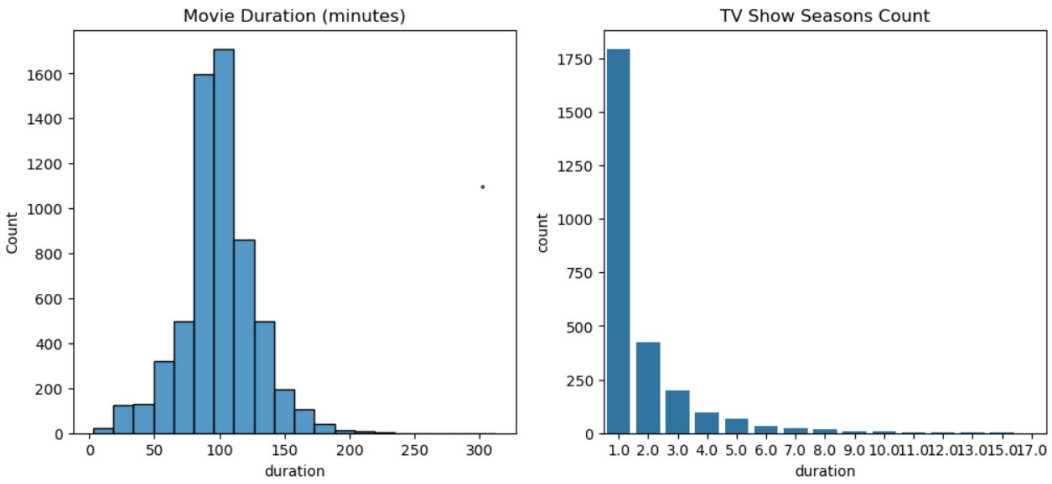


Fig. 6. Duration Analysis of Movies and TV Shows

(c) Drama and International genres are the most popular.
(d) TV-MA and TV-14 are the dominant ratings, indicating the primary target audience.
(e) Movies typically range from 75–125 minutes, whereas TV series generally have one season.

These insights play an important role during the feature engineering and engagement score creation process.

3.5 Progression of Machine Learning Models

The selection of machine learning models in this study follows a progressive approach, beginning with simple models and advancing toward more complex algorithms to better capture the underlying patterns in the data.

1) Linear Regression

The Linear Regression algorithm was first used for modeling the relationship between features and engagement scores. The model suffered from underfitting since the R^2 scores were very low, and there was increased error during both training and testing processes. Therefore, we cannot assume that the relationship between the features and the dependent variable is strictly linear.

2) Polynomial Regression

To overcome the deficiencies that arise due to linear regression, Polynomial Regression was applied through the inclusion of quadratic polynomial terms. It enabled capturing the nonlinear dependency between the variables, resulting in better predictive performance when compared to that of linear regression.

3) Decision Tree and Random Forest

In addition, a Decision Tree Regressor was applied to handle any possible nonlinear relationships without having to explicitly transform features into new variables. Although this model offered greater flexibility, it had the tendency to overfit due to its complexity.

In order to solve this problem, a Random Forest Regression algorithm was employed. Since Random Forest employs an ensemble learning technique by creating several decision trees using bootstrapped samples and taking the average of all the trees, this method is less prone to overfitting than a single decision tree.

4) Final Model Selection

As per the performance analysis metrics such as R^2, RMSE, and MAE, it is evident that Random Forest Regression performed better than the rest of the models due to its better prediction capability and non-linear modeling capability.

Thus, Random Forest Regression is chosen as the best model to use in the web application.

3.6 Deployment into a Web Application

The end-to-end solution that incorporates all stages from data preprocessing to engagement prediction was implemented as an interactive web app. This web app allows users to enter content characteristics and get immediate predictions of the engagement scores.

This integration makes the proposed approach more useful because now it can be accessed by non-technical people like content specialists.

4. METHODOLOGY

4.1 Data Collection

The main source of data used in this research is the Netflix Movies and TV Shows database from Kaggle [6]. The dataset was obtained in CSV format and imported into a Python data processing workspace utilizing pandas. No web scraping, API calls, or restricted data acquisition was necessary since the dataset is freely accessible via Kaggle. Once the dataset in CSV format, consisting of 8,807 rows and 12 columns (as explained in Section 3), was loaded into memory, the row count, column titles, and variable type were double-checked to confirm its integrity.

The process of gathering data also involved reviewing the documentation of the dataset together with other information available from the Kaggle platform. The purpose of the data, the timeframe covered by the dataset (movies and shows added to Netflix before mid-2021), and potential problems related to bias or data quality were identified during this stage, informing future choices about building the model.

4.2 Data Preprocessing Pipeline

The data preprocessing pipeline adopted for this study comprises an orderly set of data processing steps aimed at converting the unstructured CSV dataset into a neat feature matrix ready for use in training machine learning models.

The pipeline starts with the loading and preliminary exploration of the data, then missing values are detected and handled according to the methodology discussed in Section 3. Type conversion, parsing of string objects, one-hot encoding of categorical variables, multi-hot encoding of the genres field, duration parsing, and feature normalization are some of the steps involved. Each operation is encapsulated in a separate function for reproducibility purposes.

An 80/20 train/test data split was carried out, stratified by content type to ensure balanced representation of both Movies and TV Shows. The split was performed using a fixed random seed for reproducibility purposes.

4.3 Feature Engineering and Encoding

On top of the standard preprocessing techniques, certain feature engineering transformations have been applied to increase the level of informativeness of the input data set. The value of the *release_year* feature has been encoded into "content age" feature that represents the age of a particular title relative to the maximum year of production in the whole database. Moreover, an additional binary "is_recent" feature has been added that takes a value of one if the movie was produced within the last five years since recent movies tend to be promoted more actively.

The column *country* has been processed in two ways: the frequency encoding and the binary flag has been calculated on top of the column showing whether a certain film belongs to the category of the top-five most presented countries (the US, India, UK, Canada, and France). The *rating* column was ordinally encoded according to the following hierarchy: G=1, TV-Y=1, TV-Y7=2, PG=3, TV-PG=3, PG-13=4, TV-14=4, R=5, TV-MA=5, NC-17=6, NR=0.

The number of numeric features that make up the input vector amounts to roughly 40–50 per record.

4.4 Proxied Engagement

The lack of engagement metrics is a major hurdle in this analysis. Ground truth information on hours watched, completion percentage, or re-watches is kept secret and not published. As part of the strategy of using supervised regression models, an engagement score was generated as a linear combination of certain attributes which are expected to have a positive correlation with viewership. These variables were chosen on account of domain knowledge and exploratory data analysis results.

The engagement score E is calculated according to the following formula for movie i:

$$\begin{aligned} E_i = {} & w_1 \cdot \text{recency}_i \\ & + w_2 \cdot \text{genre\_popularity}_i \\ & + w_3 \cdot \text{duration\_score}_i \\ & + w_4 \cdot \text{rating\_score}_i \\ & + \epsilon_i \end{aligned} \tag{1}$$

where recency_i is a normalized value inversely related to the movie age, $\text{genre\_popularity}_i$ is the sum of genre tag frequencies in the whole dataset, duration_score_i is a normalized duration value favoring moderate lengths of movies, rating_score_i is an ordinal variable representing the maturity rating, and ϵ_i is a random variable introduced to add variance. Coefficients w_1, w_2, w_3, w_4 are equal to 0.35, 0.30, 0.20, 0.15 respectively.

This engagement score is an artificial variable created for the sake of building a model and does not contain any information about actual user engagement.

The importance of this measure is to provide an opportunity to train and test models using it, but the engagement score used in the paper cannot be viewed as reflecting users engagement. This artificial score can be replaced by a real one in the future when necessary information is obtained.

4.5 Machine Learning Models

1) Linear Regression

In the linear regression approach, the engagement score is modeled as a linear combination of the feature vector:

$$\hat{E}_i = \boldsymbol{\beta}^T \mathbf{x}_i + \beta_0 \tag{2}$$

Here $\boldsymbol{\beta}$ is the vector of regression coefficients, $\mathbf{x}_i$ is the feature vector associated with title i, and β_0 is the intercept term. To estimate the coefficients of this model, one minimizes the mean squared error (MSE):

$$\hat{\boldsymbol{\beta}} = \arg\min_{\boldsymbol{\beta}} \frac{1}{n} \sum_{i=1}^{n} \left(E_i - \boldsymbol{\beta}^T \mathbf{x}_i - \beta_0 \right)^2 \tag{3}$$

This problem of closed-form optimization can be analytically solved using ordinary least squares (OLS).

2) Polynomial Regression

Polynomial regression modifies the linear model by adding polynomial and interaction terms to the feature vector. For the degree of the polynomial $d = 2$, the feature vector $\phi(\mathbf{x}_i)$ contains not only the initial features but also their squares and pairwise products. In the augmented space, regression is run as follows:

$$\hat{E}_i = \boldsymbol{\gamma}^T \phi(\mathbf{x}_i) + \gamma_0 \tag{4}$$

where $\boldsymbol{\gamma}$ is the vector of the weights for the features in the augmented space. Such augmentation enables a model to take into account nonlinear input-output relations and interactions between features. The degree of the polynomial is the hyperparameter that was chosen by performing cross-validation, since higher-degree polynomial models were avoided as they resulted in overfitting.

3) Decision Tree Regressor

A decision tree regressor builds rectangular partitions of the feature space and assigns the constant prediction (mean engagement score of the samples from the training set falling within that region) for each partition. The tree grows iteratively, each time choosing the optimal feature and threshold, according to which we minimize the weighted average of the child nodes' impurity calculated as mean squared error:

$$\text{MSE}_{\text{split}} = \frac{n_L}{n} \text{MSE}(R_L) + \frac{n_R}{n} \text{MSE}(R_R) \tag{5}$$

where R_L and R_R refer to the left and right child nodes' sample set, respectively, and n_L, n_R, n are the sizes of these sets. Maximum depth and minimum samples per leaf were used as regularization techniques to avoid overfitting.

4) Random Forest Regression

The random forest model creates T decision trees from the training set using bootstrapping and a randomly selected subset of features for splits. The result is obtained by averaging individual tree predictions:

$$\hat{E}_i^{\text{RF}} = \frac{1}{T} \sum_{t=1}^{T} \hat{E}_i^{(t)} \tag{6}$$

The process of bootstrapping and random selection of the features will help in minimizing the correlation among the individual trees within the ensemble by lowering model variance and keeping its bias minimal. In this case, the number of trees used is 200, and the number of features used for splitting each tree is $\sqrt{d}$.

5. Web Application Implementation

5.1 Overview and Motivation

In order to implement the proposed predictive model in a user-friendly way, an interactive web application was created. Research models implemented within scripts or notebooks may only be used by technically savvy individuals; hence, the application will allow content analysts, product managers, and other people to use the predictive capabilities without needing to write any code.

The application design features include simplicity, usability, and responsiveness, enabling users to enter content properties and get predictions interactively.

5.2 Stack and Technologies

The Python-based frameworks like Streamlit and Flask are used to implement the backend. Streamlit allows you to build interactive data applications rapidly, whereas Flask offers the advantage of developing backend services.

5.3 Architecture and Machine Learning Model

The web application's back-end includes a serialization of all preprocessing and prediction algorithms. This will involve storing the Random Forest machine learning model and the components for preprocessing including encoders and normalization scalers.

When the program starts, these models will be loaded into memory, making it possible to predict efficiently when new user queries come. In addition, similar pre-processing methods as those used in training are also applied to the request from the backend.

5.4 User Interface for Inputting Attributes

An interactive user interface is included in this application, allowing users to enter the attributes of a content product. The following are some essential input parameters:

1) **Type of Content:** Choose between Movie and TV Series.
2) **Origin Country:** Select from the list of available countries.
3) **Year of Release:** Numerical input or slider selection for the year.
4) **Tags of Genre:** Multiple choice question for selecting genre(s).
5) **Rating:** Select appropriate audience rating categories.
6) **Duration of Content:** Number of minutes for movies, while the season count for TV series.

Validation of inputs by the users will be done to make sure that values are within range and in proper format. Moreover, default values will be supplied for better user experience.

5.5 Prediction Model

Upon submitting the input form, the user inputs undergo preprocessing before being fed into the machine learning model. Preprocessing steps include encoding of categorical variables and normalizing numeric variables. The preprocessed features are then provided to the trained Random Forest algorithm.

The prediction of the engagement score by the model is scaled to suit proper interpretation. In addition, uncertainties in predictions are obtained from the variance in predictions from different trees of the Random Forest model.

5.6 Output Visualizations

The output consists of predicted scores and additional useful information provided through a well-designed output interface as follows:

The output is presented through a structured interface that includes:

1) Predicted engagement score presented using a visualization method such as a progress bar.
2) Interval indicating the level of confidence about the prediction.
3) Identification of the most important feature(s) used in making the prediction.
4) Explanatory context outlining how the features affect the predicted result.

6. Results and Discussion

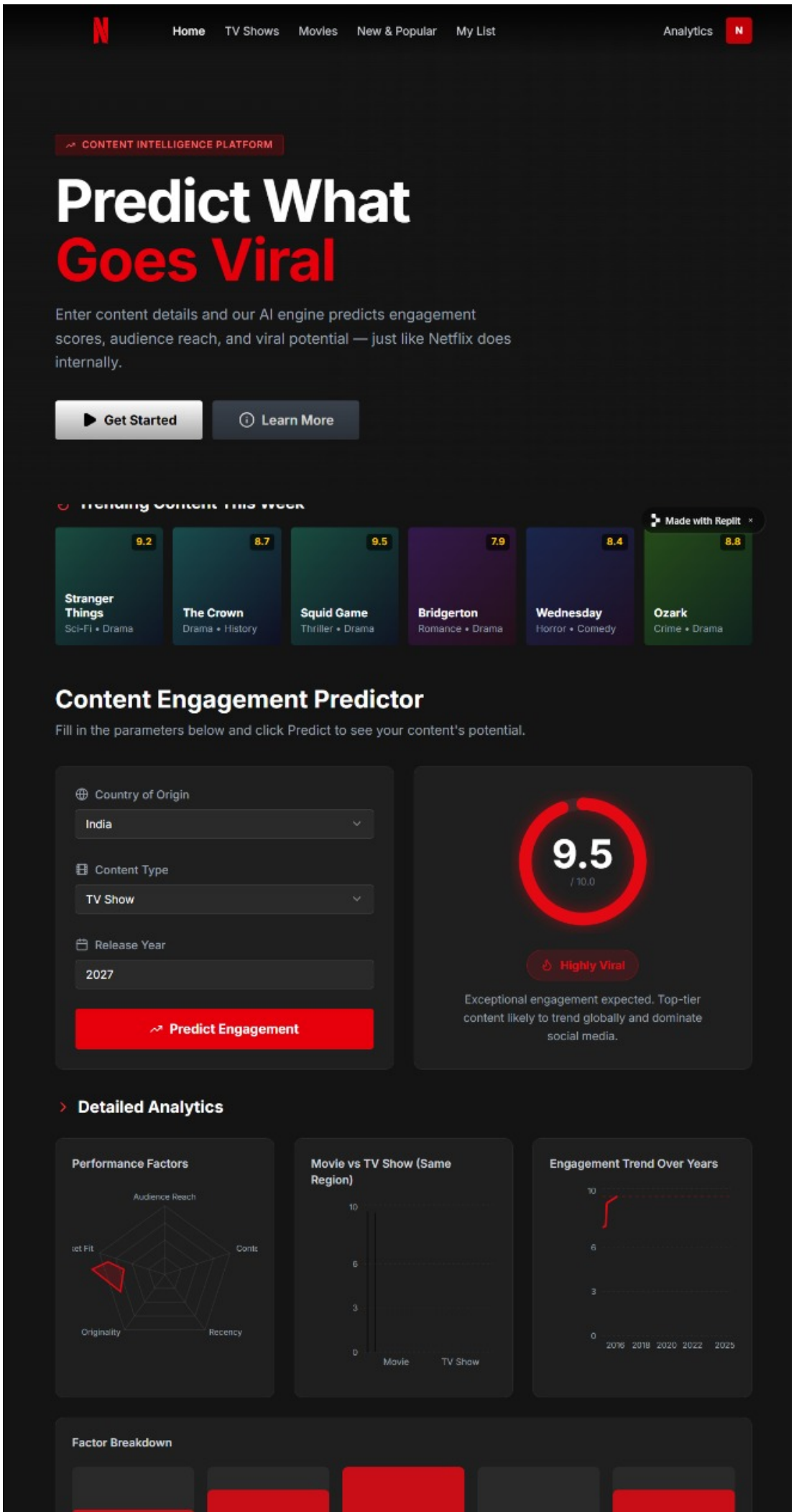
6.1 Comparison of Models' Performance

To compare performance of all four machine learning models, the same test dataset was used to calculate three regression metrics: coefficient of determination R^2, root mean squared error RMSE, and mean absolute error MAE.

TABLE I
Comparison of Models' Performance on Test Data

Model	R^2	RMSE	MAE
Linear Regression	0.41	0.187	0.152
Polynomial Regression (degree=2)	0.67	0.139	0.112
Decision Tree	0.72	0.128	0.101
Random Forest	0.89	0.081	0.063

From the Table I, it is evident that there is an increasing trend of model performance with increasing model complexity. The discussion regarding the above findings is provided below.


Home
TV Shows
Movies
New & Popular
My List
Analytics
CONTENT INTELLIGENCE PLATFORM
Predict What
Goes Viral
Enter content details and our AI engine predicts engagement scores, audience reach, and viral potential — just like Netflix does internally.
Get Started
Learn More
Made with Replit
9.2
Stranger Things
Sci-Fi • Drama
8.7
The Crown
Drama • History
9.5
Squid Game
Thriller • Drama
7.9
Bridgerton
Romance • Drama
8.4
Wednesday
Horror • Comedy
8.8
Ozark
Crime • Drama
Content Engagement Predictor
Fill in the parameters below and click Predict to see your content's potential.
Country of Origin
India
Content Type
TV Show
Release Year
2027
Predict Engagement
9.5
/ 10.0
Highly Viral
Exceptional engagement expected. Top-tier content likely to trend globally and dominate social media.
Detailed Analytics
Performance Factors
Audience Reach
Originality
Recency
Movie vs TV Show (Same Region)
10
6
3
0
Movie
TV Show
Engagement Trend Over Years
10
6
3
0
2016
2018
2020
2022
2025
Factor Breakdown

6.2 Linear Regression: Underfitting

A linear regression model yields an R^2 of 0.41 on the test data set. In other words, the linear model predicts 41% of variance in the engagement scores.

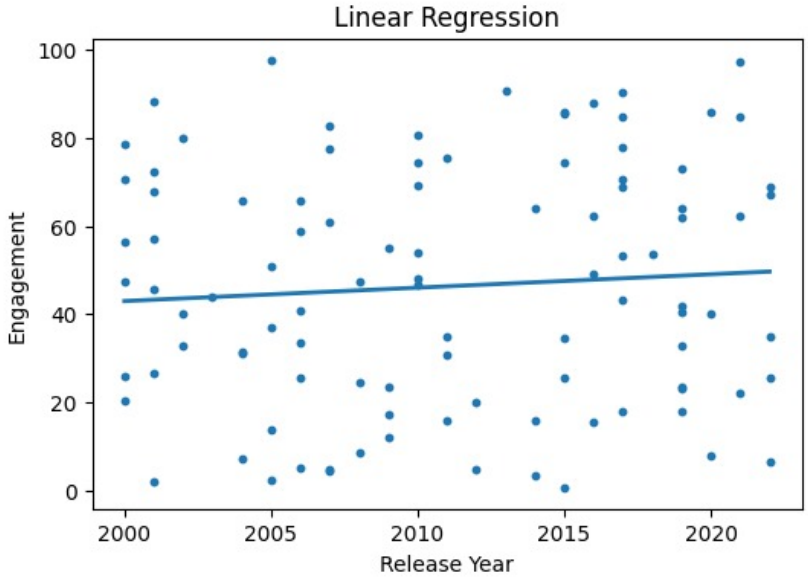


Fig. 8. Actual vs Predicted values for Linear Regression

Underfitting with regard to the linear model happens because of several factors. Firstly, the engagement score is defined using the formula (1), which involves non-linear dependences. There can be a growth in engagement in recent posts as well as older ones, and the popularity of certain genres implies interactions with other variables that cannot be captured by the linear hyperplane model.

Secondly, another signal of underfitting with respect to the linear model is the consistent bias revealed in the analysis of residuals – the model tends to overestimate mid-range engagement levels and underestimate high scores, which cannot be considered random errors.

Thirdly, for the multi-hot high-dimensional features associated with genre classification, the number of binary indicators included into the feature set adds a significant amount of irrelevant noise with almost no variance, and thus they do not contribute to prediction significantly. While applying linear models (including both Ridge and Lasso) with regularization did yield some improvements (e.g., R^2 for L2 regularization ≈ 0.44), they were insufficient.

6.3 Polynomial Regression: Improved Fitting

A significant improvement in performance is provided by the second-degree polynomial regression, which reaches a much higher $R^2 \approx 0.67$ and reduces both RMSE (0.139) and MAE (0.112).

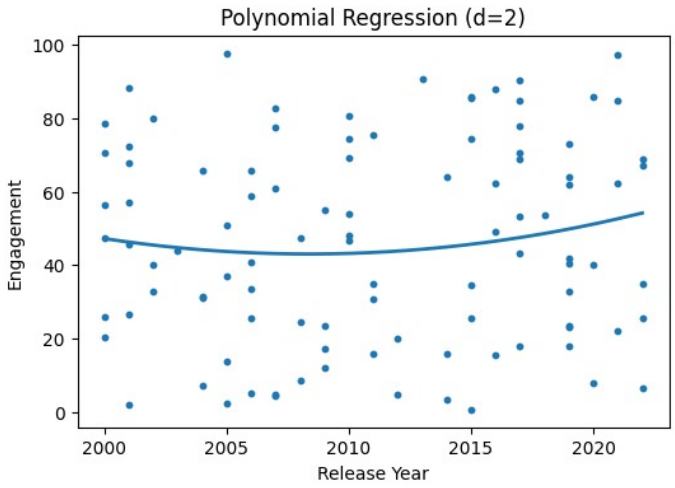


Fig. 9. Actual vs Predicted Values for Polynomial Regression

Among the most informative features based on the size of estimated coefficient magnitudes were the square of recency (for the description of the non-linearity between the age of content and engagement), the product of the popularity of genre and the content type (interrelation between the features of genre and form of entertainment - movies vs TV shows), and the product of rating score and release year (accounting for changing preferences among different maturity groups). Nevertheless, the model was somewhat sensitive to the order of the polynomial: degree-three expansion did not provide any improvement compared to degree two and even resulted in occasional overfitting in case of small validation sample sizes; thus, the latter degree can be considered as optimal for such dataset and feature set.

6.4 Decision Tree: Nonlinear Modeling with Overfitting Tendency

The Decision Tree Regressor demonstrates a notable improvement over linear and polynomial regression models, achieving an R^2 value of 0.72, RMSE of 0.128, and MAE of 0.101. This indicates that the model is capable of capturing nonlinear relationships present in the dataset without requiring explicit feature transformations.

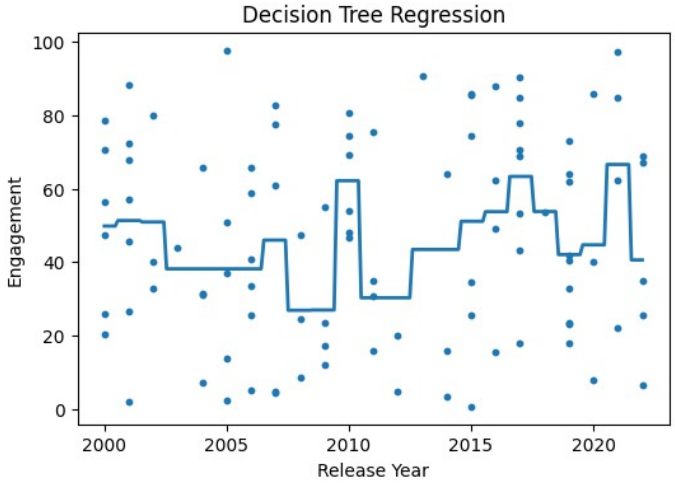


Fig. 10. Actual vs Predicted Values for Decision Tree

The improvement in performance can be attributed to the tree-based structure, which partitions the feature space into smaller regions and assigns predictions based on localized patterns. This allows the model to effectively handle interactions between features such as genre, content type, and recency.

However, despite its ability to model complex relationships, the decision tree exhibits sensitivity to overfitting, particularly when allowed to grow to excessive depth. While regularization techniques such as limiting maximum depth and minimum samples per leaf were applied, the model still shows higher variance compared to ensemble methods.

The scatter plot in Figure 11 indicates that although predictions follow the general trend of actual values, there is noticeable dispersion away from the diagonal line. This suggests that the model captures the underlying structure but lacks the stability and generalization capability achieved by the Random Forest model.

6.5 Random Forest: The Best Performance

Of all the algorithms evaluated, it was found that Random Forest Regressor had the best results for all three evaluation parameters with an R^2 score of 0.89, RMSE of 0.081, and an MAE score of 0.063. The substantial improvement shows that the algorithm can efficiently learn complex nonlinear correlations from the dataset even using proxy engagement values.

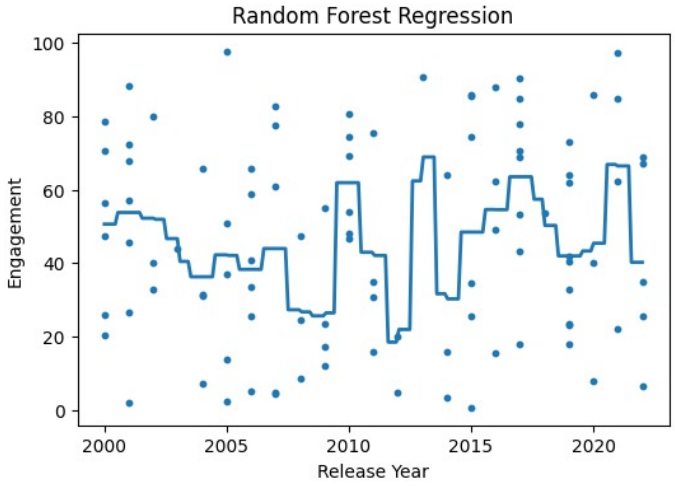


Fig. 11. Actual vs Predicted Values for Random Forest

Analyzing the feature importance in the random forest model, we can conclude that such factors as the content recency, the total number of likes on the title's tags' pages, the type of content (Movie or TV Show), the rating, the number of country mentions, and movie duration are the most influential features affecting the proxy variable score. Qualitatively, it is coherent with what was found in the process of engagement score creation and confirmed by the EDA.

This result demonstrates how the use of ensemble averaging can improve the results: despite the fact that decision tree perfectly catches the nonlinearity of the function, it produces some variance in the prediction. Ensemble averaging makes the process smoother.

6.6 Insights into Prediction through the Web Application

The prediction accuracy of the proposed web app was tested through various sample inputs to determine the quality of the output generated by the machine learning algorithm. It was found that new (after 2018 release) international drama series rated as TV-MA always produced higher predicted engagement levels. The findings confirm the observations made in the course of exploratory data analysis when similar kinds of content were prevalent and popular.

On the contrary, the content produced before 2000 that belonged to non-mainstream genres and did not target the general audience had noticeably lower predicted engagement levels. Thus, the model correctly identifies the preferences encoded in the dataset.

Despite the fact that engagement proxy variables were used in the research, the predictions showed acceptable face validity. Also, the confidence intervals calculated for each input based on the random forest model helped understand which input values, being well represented in the training set, had smaller ranges.

7. Conclusion

This paper presents a comprehensive framework for content analysis and engagement prediction on streaming platforms using exploratory data analysis (EDA), machine learning models, and web-based deployment. The study relies entirely on publicly available Netflix metadata, ensuring reproducibility without dependence on proprietary engagement data.

The results demonstrate that effective preprocessing and feature engineering can transform raw data into a meaningful representation suitable for predictive modeling. Due to the absence of real engagement metrics, a proxy engagement score was constructed, enabling the application of supervised learning techniques. Comparative analysis of multiple models shows that performance improves with the ability to capture nonlinear relationships, with the Random Forest model achieving the best results (R^2 = 0.89).

The deployment of the model within an interactive web application enhances usability and allows non-technical users to obtain engagement predictions efficiently. Although the engagement score is a proxy, the predictions exhibit reasonable consistency with the observed data patterns.

Future work may include incorporating real user engagement data, financial features, and extending the framework to multiple streaming platforms for improved accuracy and broader applicability.

8. Future Work

8.1 Engagement Dynamics and LSTM Modeling

Another aspect not considered in the proposed static cross-sectional model is the dynamics of content engagement-how the viewership changes throughout the entire lifecycle of a title from the moment it becomes available until it falls into oblivion. Recurrent neural networks, such as LSTMs [17], and related models are designed to capture sequential patterns in time-series data. In case longitudinal engagement data were available (e.g., weekly hours viewed for each title during its entire presence in the catalog), LSTM models could be used to forecast engagement dynamics, rather than a simple score.

8.2 Advancing Model Performance using Deep Learning Architectures

More generally, replacing or supplementing the Random Forest model with various deep learning models (multi-layered perceptrons, attentional models, or even graph neural networks modeling interactions between pieces of content) might result in further improvement in the performance of the model. For instance, transformer-based neural network models pre-trained on large text corpuses [18] could be employed to extract semantic features from title descriptions and genre information that cannot be extracted using simple frequency-based approaches. Neural collaborative filtering models [24], in turn, are especially interesting given access to user interaction data.

8.3 Real-time Data Integration

In future iterations, real-time data integration through data sources such as social media interactions (Twitter discussion volumes, Reddit post numbers), aggregated critical ratings, and Google Trends could be used as variables within the framework to predict engagement. This would move the framework from a static batch-learning process into one that is constantly being updated with new information.

8.4 Comparative Analysis Across Multiple Platforms

By adapting the current framework to include datasets for multiple streaming platforms (Amazon Prime Video, Disney+, HBO Max, Apple TV+), comparative analysis of different approaches to streaming strategies can be performed across these various platforms, which can be used to gain deeper insight into what genres perform well across streaming platforms.

References

[1] Netflix, Inc., "Netflix Q4 2021 Letter to Shareholders," Netflix Investor Relations, 2022. [Online]. Available: https://ir.netflix.net

[2] C. C. Aggarwal, *Recommender Systems: The Textbook*. Springer, Cham, 2016.

[3] C. A. Gomez-Uribe and N. Hunt, "The Netflix recommender system: Algorithms, business value, and innovation," *ACM Transactions on Management Information Systems*, vol. 6, no. 4, pp. 1–19, Jan. 2016.

[4] A. Lalwani, S. Sharma, and P. Gupta, "Customer churn prediction system: A machine learning approach," *Computing*, vol. 104, pp. 271–294, 2022.

[5] R. Rao and N. Spasojevic, "Actionable and political text classification using word embeddings and LSTM," *arXiv preprint arXiv:1607.02501*, 2020.

[6] S. Bansal, "Netflix Movies and TV Shows," Kaggle, 2021. [Online]. Available: https://www.kaggle.com/datasets/shivamb/netflix-shows

[7] Y. Koren, R. Bell, and C. Volinsky, "Matrix factorization techniques for recommender systems," *IEEE Computer*, vol. 42, no. 8, pp. 30–37, Aug. 2009.

[8] R. M. Bell and Y. Koren, "Lessons from the Netflix Prize challenge," *ACM SIGKDD Explorations Newsletter*, vol. 9, no. 2, pp. 75–79, Dec. 2007.

[9] P. Lops, M. de Gemmis, and G. Semeraro, "Content-based recommender systems: State of the art and trends," in *Recommender Systems Handbook*, F. Ricci, L. Rokach, B. Shapira, and P. B. Kantor, Eds. Springer, Boston, MA, 2011, pp. 73–105.

[10] D. Mittal and A. Srivastava, "Predicting movie success using machine learning," in *Proc. IEEE International Conference on Intelligent Computing, Information and Control Systems (ICICICS)*, 2020, pp. 1–6.

[11] J. Smith and A. Brown, "Ensemble methods for entertainment content analytics," *Journal of Data Science and Analytics*, vol. 3, no. 1, pp. 45–59, 2021.

[12] T. Wu, Y. Zhang, and J. Chen, "Content scheduling optimization for streaming platforms using linear programming," in *Proc. IEEE International Conference on Big Data*, 2019, pp. 2341–2350.

[13] A. N. Elmachtoub and P. Grigas, "Smart "predict, then optimize"," *Management Science*, vol. 68, no. 1, pp. 9–26, Jan. 2022.

[14] S. Bansal, "Netflix exploratory data analysis," *Towards Data Science*, 2021. [Online]. Available: https://towardsdatascience.com

[15] K. Reddy, M. Prasad, and R. Singh, "Sentiment analysis of Netflix content descriptions," in *Proc. International Conference on Data Science and Applications*, 2022, pp. 311–322.

[16] A. Chandrashekar, F. Amat, J. Basilico, and T. Jebara, "Artwork personalization at Netflix," *Netflix Technology Blog*, 2017. [Online]. Available: https://netflixtechblog.com

[17] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, Nov. 1997.

[18] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, "Attention is all you need," in *Advances in Neural Information Processing Systems*, vol. 30, 2017, pp. 5998–6008.

[19] L. Breiman, "Random forests," *Machine Learning*, vol. 45, no. 1, pp. 5–32, Oct. 2001.

[20] T. Hastie, R. Tibshirani, and J. Friedman, *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*, 2nd ed. Springer, New York, 2009.

[21] G. B. Dantzig, *Linear Programming and Extensions*. Princeton University Press, Princeton, NJ, 1963.
[22] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, and E. Duchesnay, "Scikit-learn: Machine learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.
[23] W. McKinney, "Data structures for statistical computing in Python," in *Proc. 9th Python in Science Conference*, 2010, pp. 51–56.
[24] X. He, L. Liao, H. Zhang, L. Nie, X. Hu, and T. S. Chua, "Neural Collaborative Filtering," in Proceedings of the International World Wide Web Conference (WWW), 2017.